

Lesson 02: Lesson End Project

Deploying a Resilient Web Application with Auto Scaling and Multi-AZ RDS

Project agenda: To design and implement a highly available web application architecture on AWS that can withstand heavy user traffic while ensuring reliability, scalability, and uninterrupted service delivery. The solution will leverage Multi-AZ Amazon RDS deployments for database high availability and fault tolerance, ensuring automatic failover in case of instance or availability zone failure.

Description: You are tasked with implementing a highly available and scalable web application architecture on AWS to ensure reliability, fault tolerance, and uninterrupted service delivery during high user traffic. The solution leverages Multi-AZ RDS for database resilience and automatic failover, Auto Scaling Groups for dynamic adjustment of application instances based on demand, and an Elastic Load Balancer for distributing incoming traffic efficiently. By combining these AWS services with monitoring, logging, and health checks, the architecture will provide a resilient, self-healing environment that can handle unexpected spikes, maintain business continuity, and optimize resource usage.

Tools required: AWS RDS, AWS Auto Scaling, EC2

Prerequisites: None

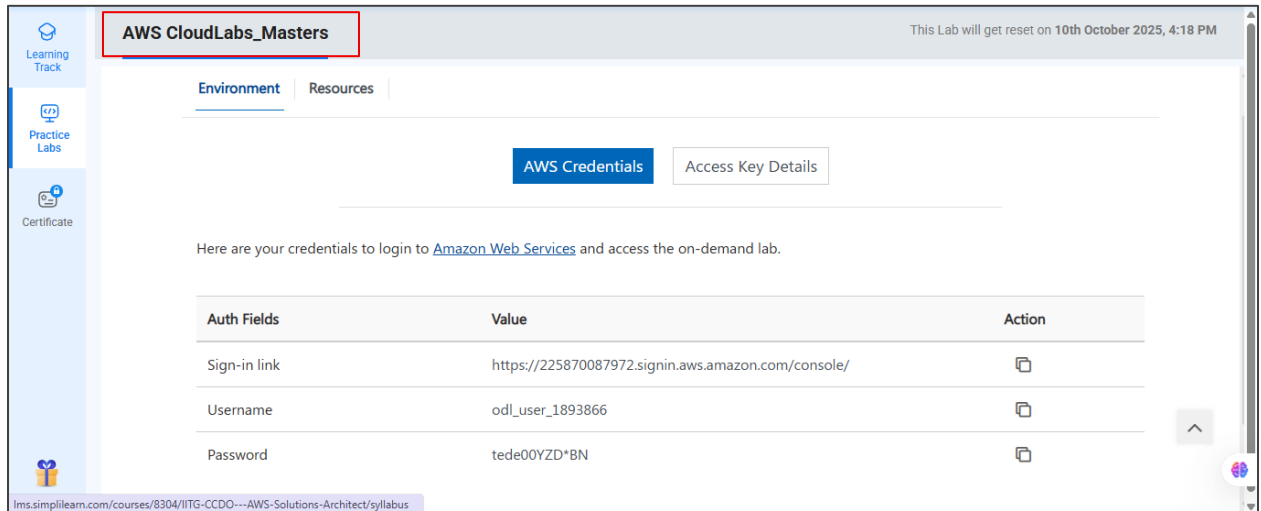
Expected deliverables: A fully implemented cloud architecture featuring a Multi-AZ RDS database and Auto Scaling-enabled application tier, an operational load-balanced web application accessible under varying traffic loads, a configuration guide detailing setup steps and architectural decisions, monitoring and alerting dashboards with CloudWatch for performance and reliability tracking, documented test scenarios demonstrating high availability and scalability under simulated traffic spikes, and a final report summarizing outcomes, performance results, and recommendations for further optimization.

Steps to be followed:

1. Create a Multi-AZ MySQL instance in the RDS service
2. Create an Auto Scaling mechanism for a Web application
3. Validate Auto Scaling and Multi-AZ mechanism

Step 1: Create a Multi-AZ MySQL instance in the RDS service

1.1 Launch AWS CloudLabs_Masters

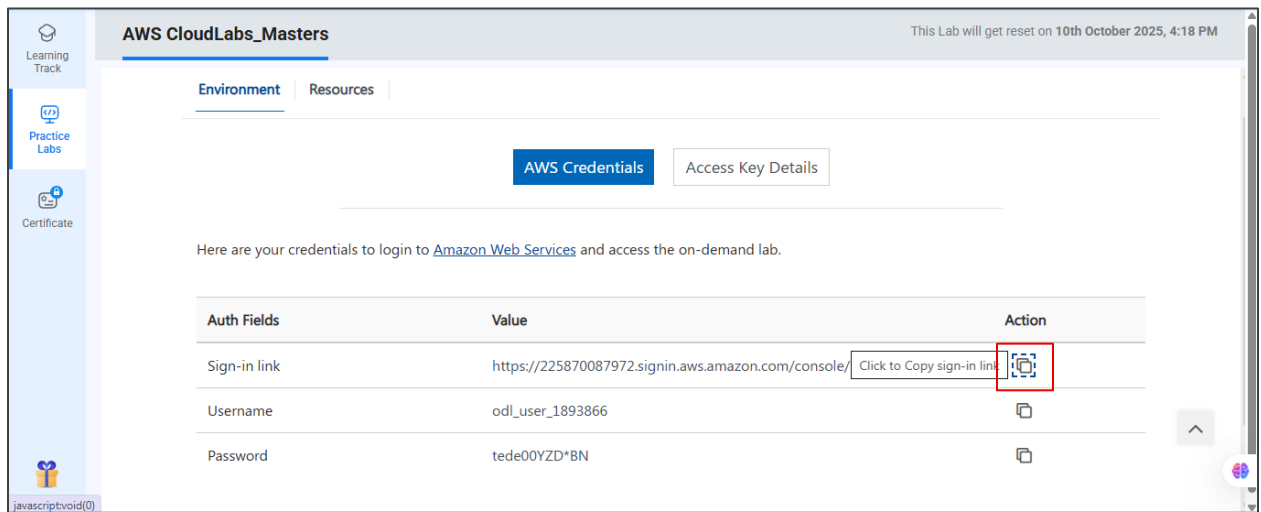


The screenshot shows the AWS CloudLabs_Masters interface. The top bar indicates the lab will reset on 10th October 2025, 4:18 PM. The left sidebar contains links for Learning Track, Practice Labs, and Certificate. The main content area is titled "Environment" and "Resources". Below this, there are buttons for "AWS Credentials" and "Access Key Details". A message states: "Here are your credentials to login to [Amazon Web Services](#) and access the on-demand lab."

Auth Fields	Value	Action
Sign-in link	https://225870087972.signin.aws.amazon.com/console/	
Username	odl_user_1893866	
Password	tede00YZD*BN	

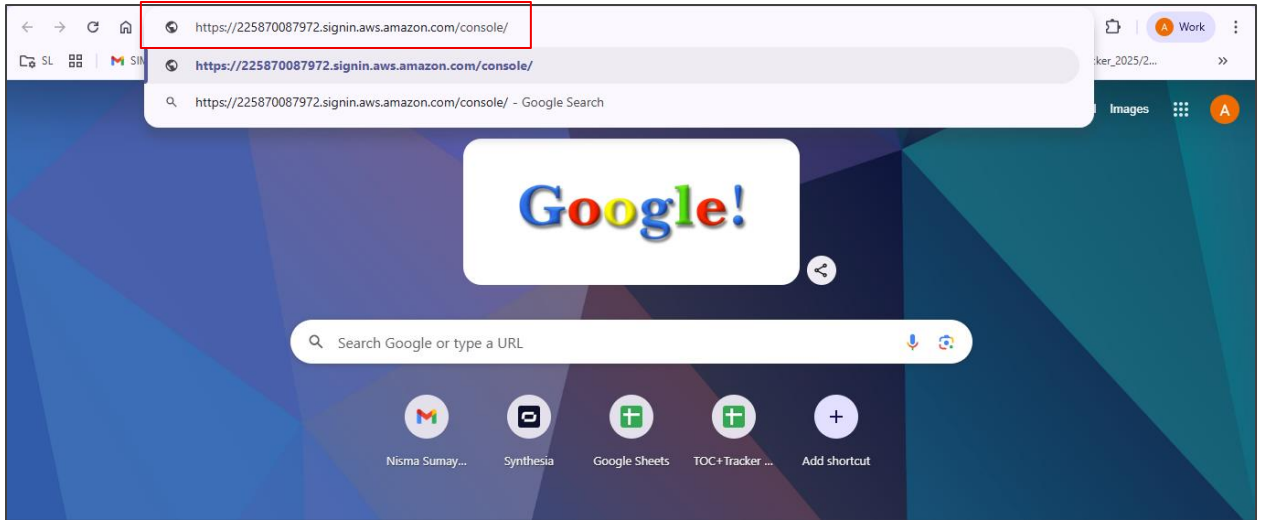
The URL bar at the bottom shows: `lms.simplilearn.com/courses/8304/IITG-CCDO---AWS-Solutions-Architect/syllabus`

1.2 Click the Copy sign-in link button

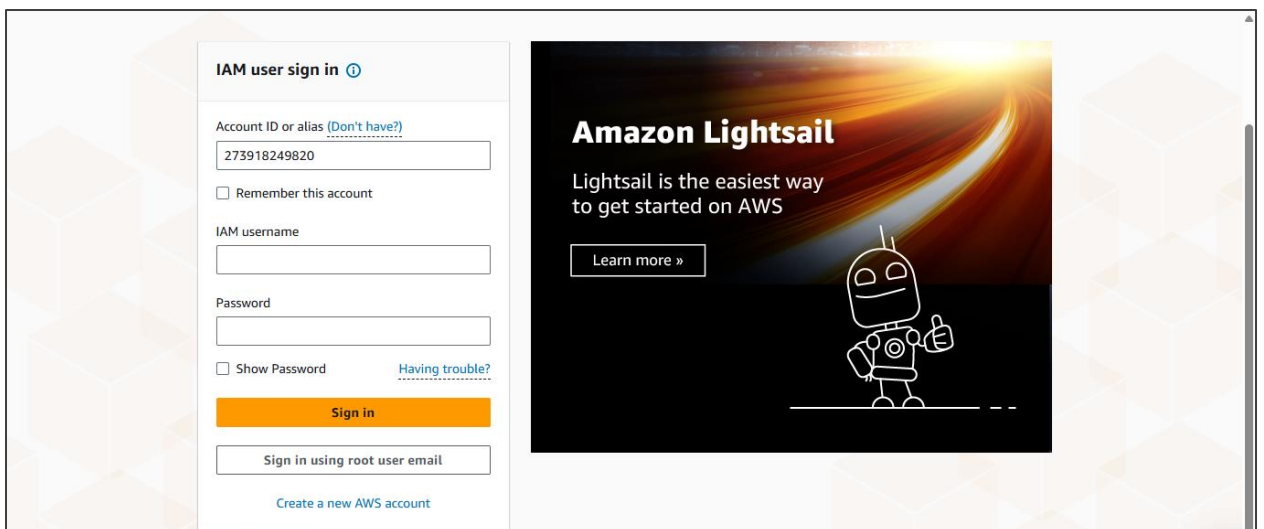


This screenshot is identical to the previous one, but with a red box highlighting the "Click to Copy sign-in link" button in the "Action" column of the table. The button is a small square with a copy icon and the text "Click to Copy sign-in link".

1.3 Paste the copied link into the browser and press enter to open the AWS console login page



The following page opens:



1.4 Click on the copy icons to copy the AWS console **Username** and **Password**

Learning Track

Practice Labs

Certificate

AWS CloudLabs_Masters

This Lab will get reset on 10th October 2025, 4:18 PM

Environment

Resources

AWS Credentials

Access Key Details

Here are your credentials to login to [Amazon Web Services](#) and access the on-demand lab.

Auth Fields	Value	Action
Sign-in link	https://225870087972.signin.aws.amazon.com/console/	
Username	odl_user_1893866	Click to Copy Username
Password	tede00YZD*BN	

Learning Track

Practice Labs

Certificate

AWS CloudLabs_Masters

This Lab will get reset on 10th October 2025, 4:18 PM

Environment

Resources

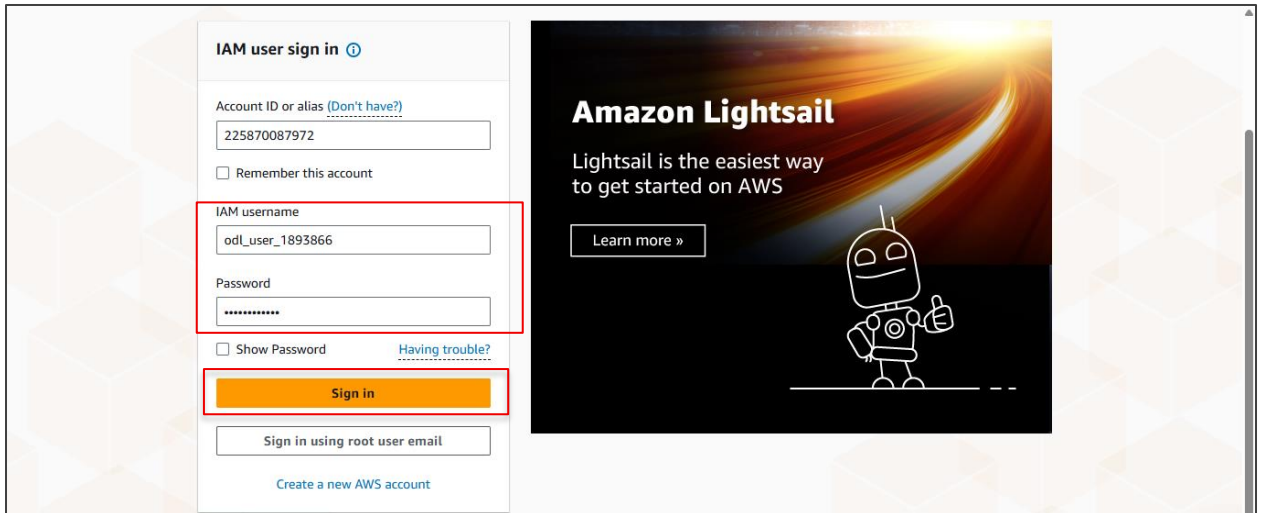
AWS Credentials

Access Key Details

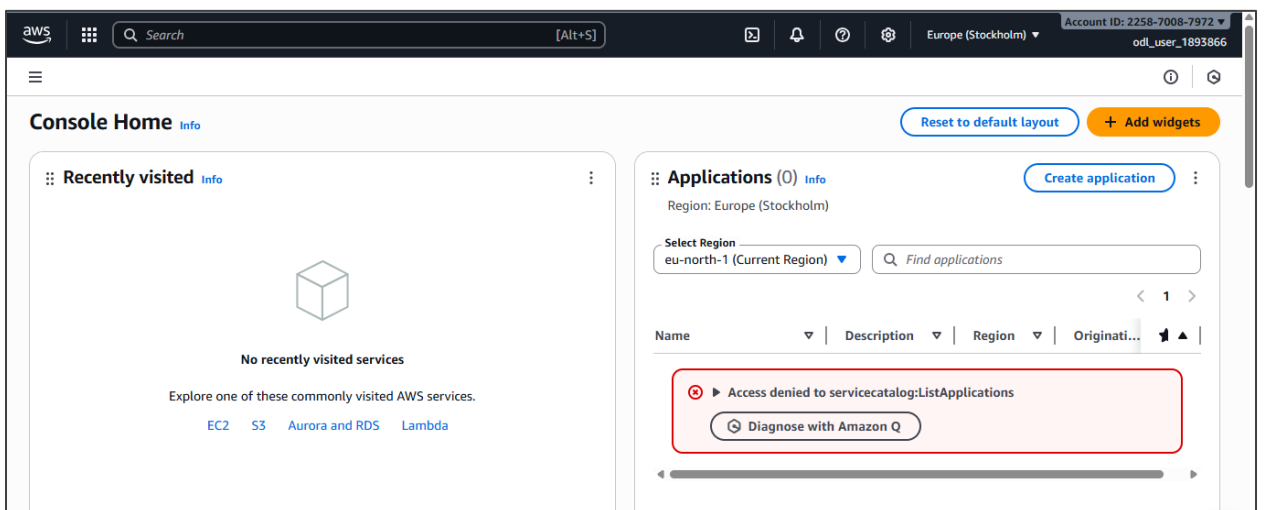
Here are your credentials to login to [Amazon Web Services](#) and access the on-demand lab.

Auth Fields	Value	Action
Sign-in link	https://225870087972.signin.aws.amazon.com/console/	
Username	odl_user_1893866	Click to Copy Password
Password	tede00YZD*BN	

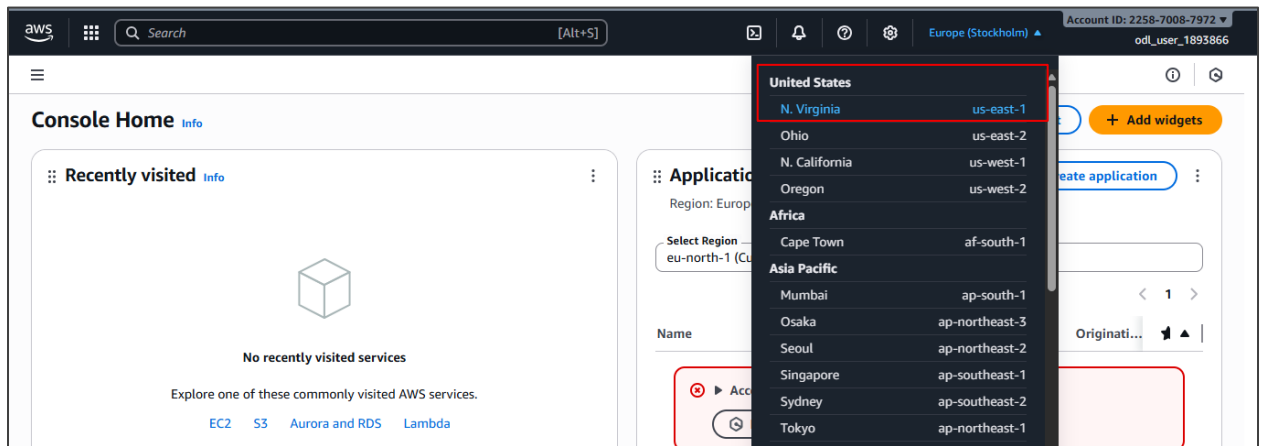
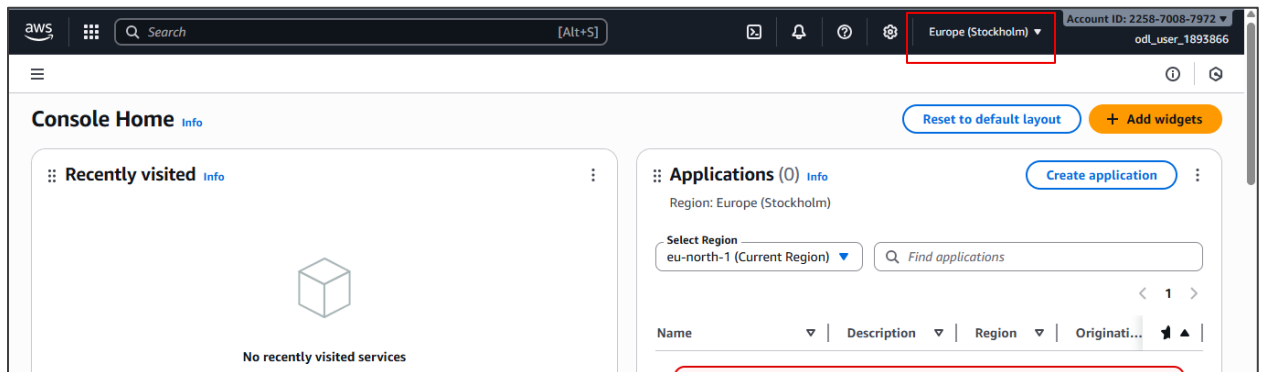
1.5 Paste the copied **IAM username** and **Password**, then click on the **Sign in** button to access the AWS Console



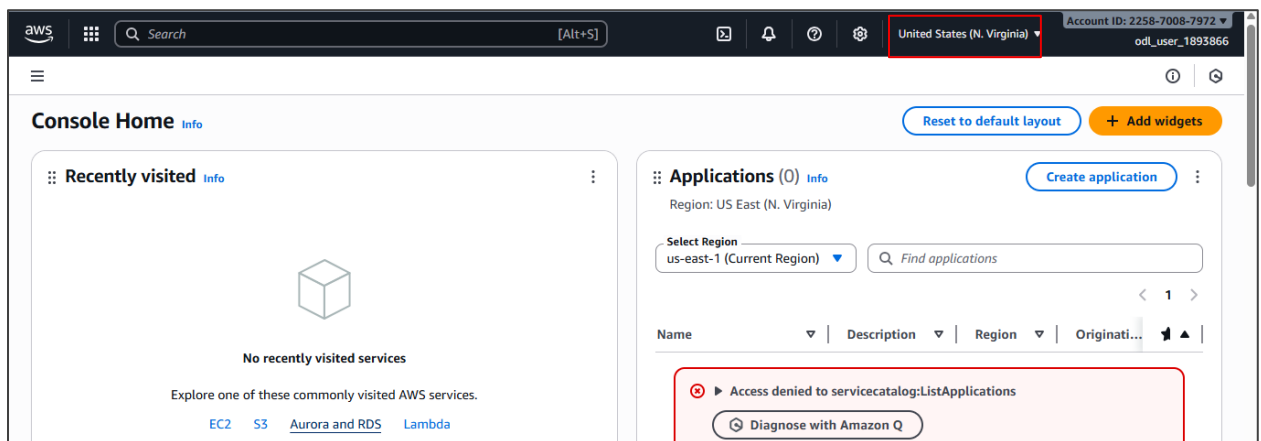
The following page appears:



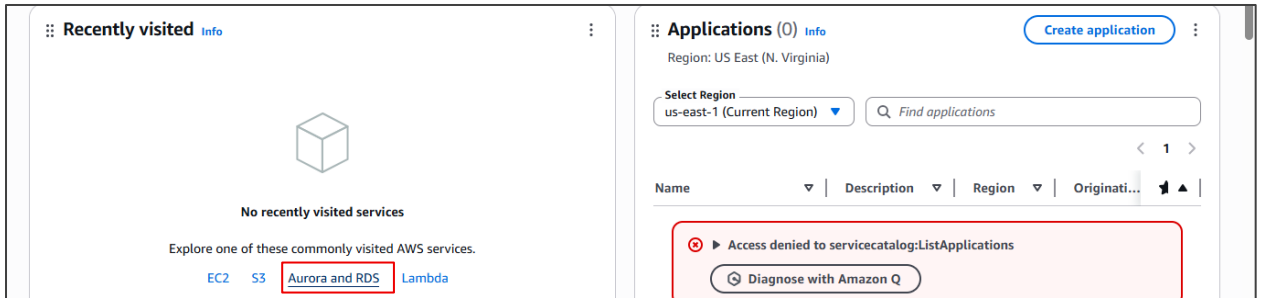
1.6 Switch the AWS region from **Europe (Stockholm)** to **United States (N. Virginia us-east-1)** from the region dropdown in the console



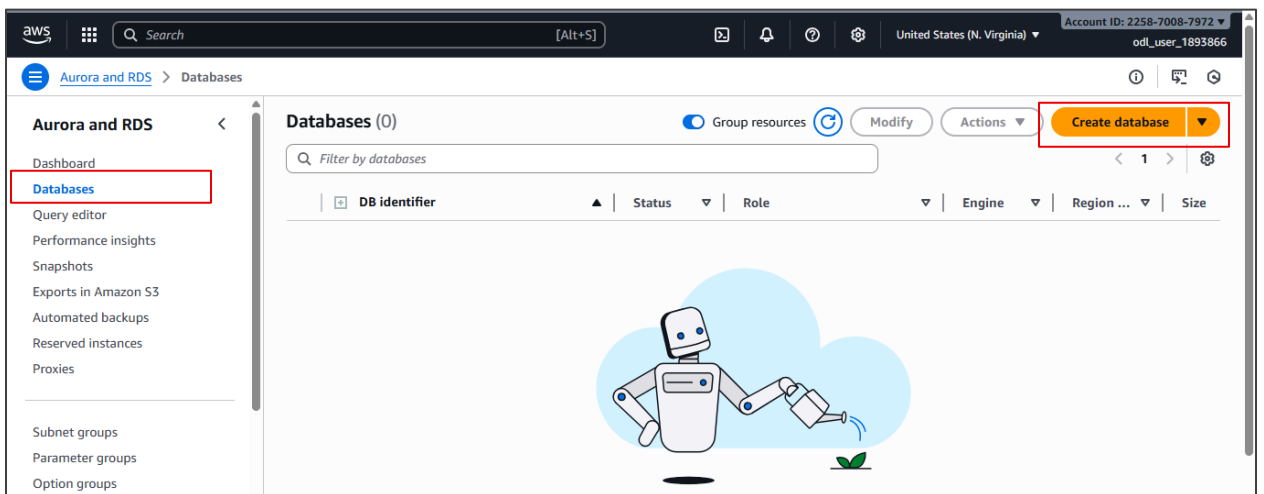
The following page appears:



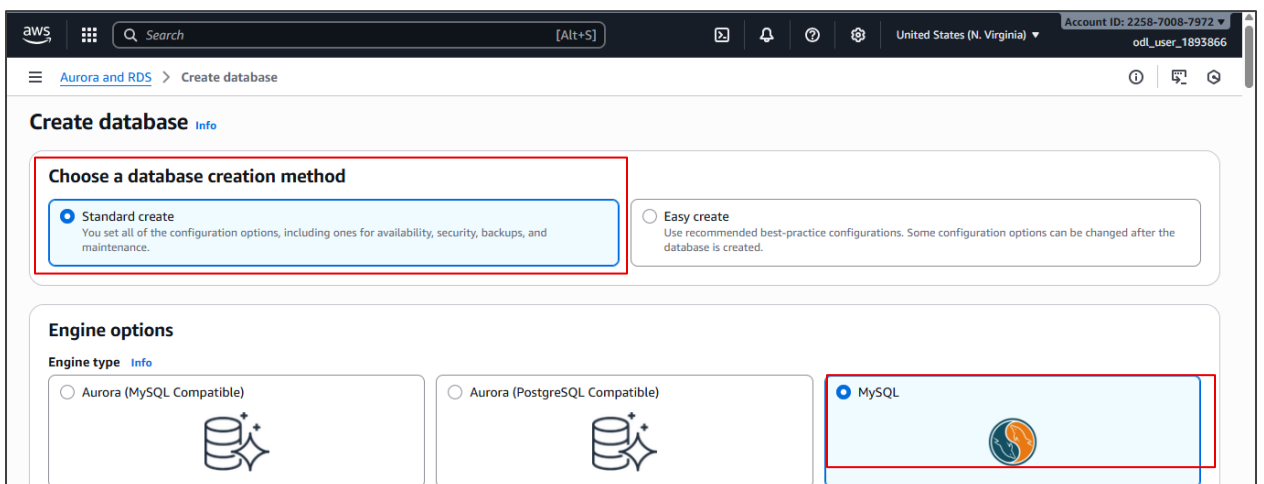
1.7 Click **Aurora and RDS**



1.8 Click **Databases**, then click **Create database**



1.9 Select **Standard create** as the database creation method and choose **MySQL** as the engine type



1.10 Select **Dev/Test** as sample template and choose **Multi-AZ DB instance deployment (2 instances)** as **Availability and durability**

The screenshot shows the AWS Aurora and RDS 'Create database' console. The 'Choose a sample template to meet your use case.' section has three options: 'Production', 'Dev/Test' (selected), and 'Sandbox'. The 'Dev/Test' option is highlighted with a red box. Below this, the 'Availability and durability' section is titled 'Deployment options'. It has three options: 'Multi-AZ DB cluster deployment (3 instances)', 'Multi-AZ DB instance deployment (2 instances)' (selected), and 'Single-AZ DB instance deployment (1 instance)'. The 'Multi-AZ DB instance deployment (2 instances)' option is highlighted with a red box. Below this, there are diagrams for each deployment option. The 'Multi-AZ DB instance deployment (2 instances)' diagram shows a primary DB instance in AZ 1 and a standby instance in AZ 2, both with write/read endpoints. The 'Single-AZ DB instance deployment (1 instance)' diagram shows a single DB instance in AZ 1 with a write/read endpoint.

1.11 Enter the **DB instance identifier** as **sourceDB** and **Master username** as **admin**, then select **Self managed** for **Credentials management**

The screenshot shows the AWS Aurora and RDS 'Create database' console, specifically the 'Settings' section. The 'DB instance identifier' field is set to 'sourceDB' and is highlighted with a red box. Below this, the 'Credentials Settings' section is expanded. The 'Master username' field is set to 'admin' and is highlighted with a red box. Below this, the 'Credentials management' section has two options: 'Managed in AWS Secrets Manager - most secure' and 'Self managed' (selected). The 'Self managed' option is highlighted with a red box.

1.14 Enter a strong master password, confirm it, and ensure the password strength indicator shows **Very strong**

The screenshot shows the 'Create database' page in the AWS Management Console. The 'Master password' section is highlighted with a red box. It includes a 'Master password' field, a 'Password strength' indicator showing 'Very strong' (with a green bar), and a 'Confirm master password' field. The 'Password strength' indicator is also highlighted with a red box.

1.15 Select **Burstable classes** and choose **db.t3.micro** as the DB instance class

The screenshot shows the 'Instance configuration' page in the AWS Management Console. The 'DB instance class' section is highlighted with a red box. It includes a 'DB instance class' dropdown menu with 'db.t3.micro' selected. The 'Burstable classes (includes t classes)' option is selected, and the 'db.t3.micro' instance class is chosen from the dropdown menu.

1.16 Select **General Purpose SSD (gp3)** as the **Storage type** and allocate **20 GB** of storage as **Allocated storage**

The screenshot shows the 'Storage' page in the AWS Management Console. The 'Storage type' section is highlighted with a red box, showing 'General Purpose SSD (gp3)' selected. The 'Allocated storage' section is also highlighted with a red box, showing '20' GB allocated. The 'Storage type' dropdown menu is set to 'General Purpose SSD (gp3)' and the 'Allocated storage' dropdown menu is set to '20'.

1.17 Enable **Public access**, enable **Create new VPC security group**, and name it as **sg_database**

Aurora and RDS > Create database

Public access [Info](#)

☒ **Yes**
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☐ **No**
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☐ Choose existing
Choose existing VPC security groups

☒ **Create new**
Create new VPC security group

New VPC security group name

sg_database

RDS Proxy
RDS Proxy is a fully managed, highly available database proxy that improves application scalability, resiliency, and security.

☐ **Create an RDS Proxy** [Info](#)
RDS automatically creates an IAM role and a Secrets Manager secret for the proxy. RDS Proxy has additional costs. For more information, see [Amazon RDS Proxy pricing](#).

1.18 Review the configuration settings and click **Create database** to launch the RDS instance

Aurora and RDS > Create database

Log exports
Select the log types to publish to Amazon CloudWatch Logs

☐ Audit log
☐ Error log
☐ General log
☐ iam-db-auth-error log
☐ Slow query log

IAM role
The following service-linked role is used for publishing logs to CloudWatch Logs.

RDS service-linked role

Additional configuration
Database options, encryption turned on, backup turned on, backtrack turned off, maintenance, CloudWatch Logs, delete protection turned off.

You are responsible for ensuring that you have all of the necessary rights for any third-party products or services that you use with AWS services.

[Cancel](#) [Create database](#)

The following page appears:

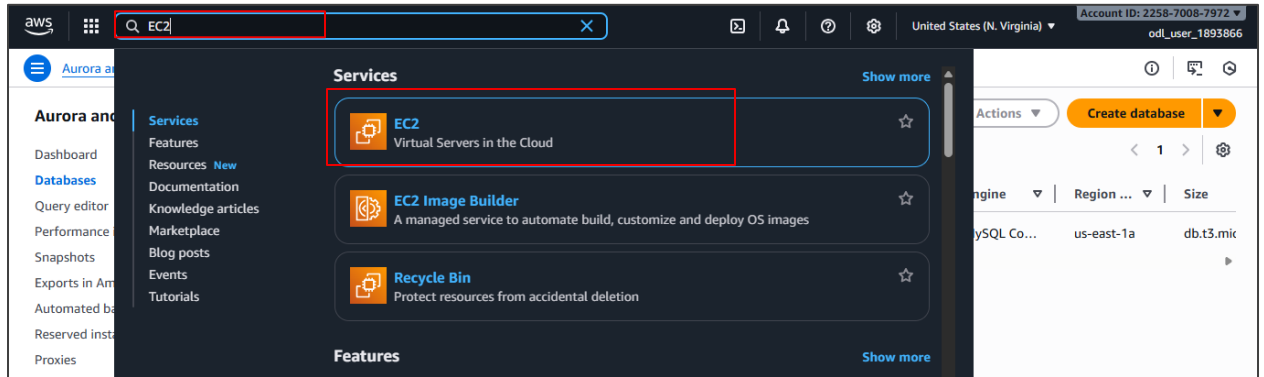
Aurora and RDS > Databases

Databases (2) [Group resources](#) [Modify](#) [Actions](#) [Create database](#)

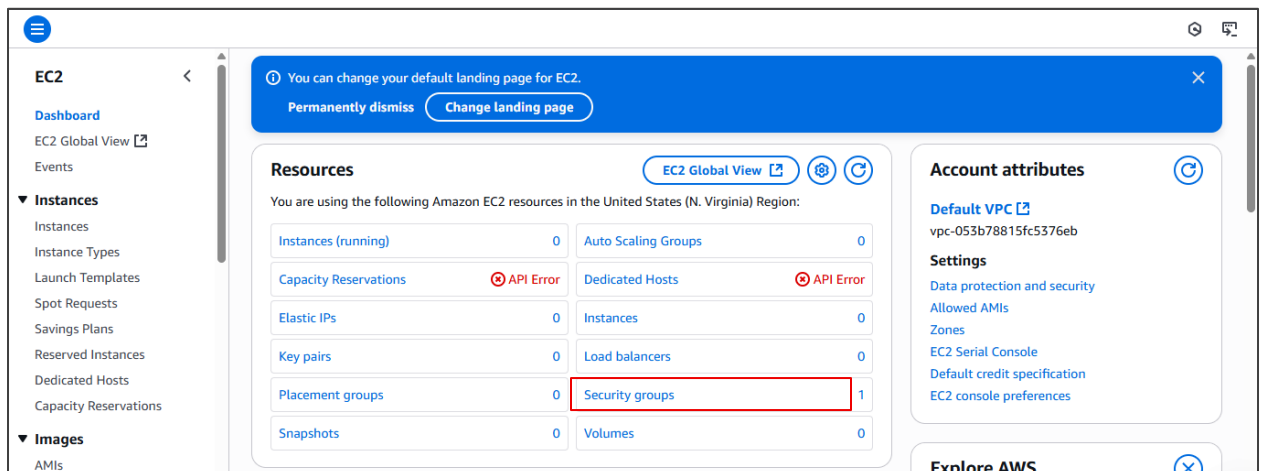
☐	DB identifier	Status	Role	Engine	Region	Size
☐	database-1	Modify...	Instance	MySQL Co...	us-east-1a	db.t3.mic
☐	sourcedb	Creating	Instance	MySQL Co...	-	db.t3.mic

Step 2: Create an Auto Scaling mechanism for the Web application

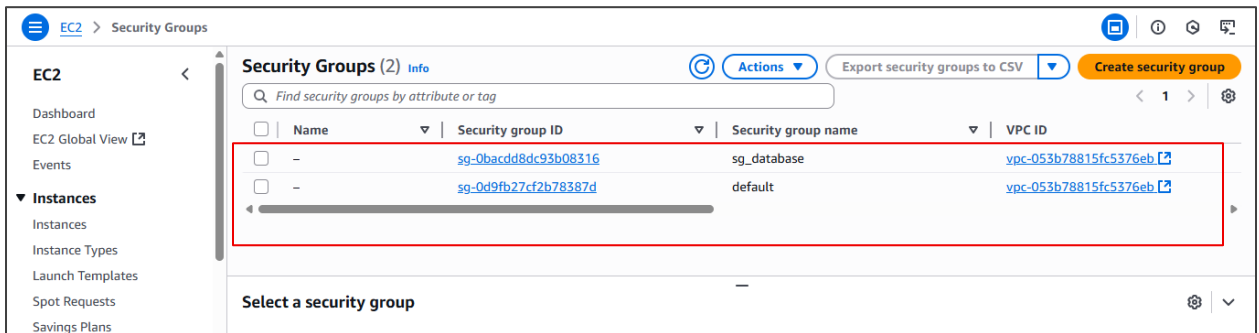
2.1 Search for EC2 in the AWS console search bar and select **EC2 Virtual Servers in the Cloud** from the **Services** list



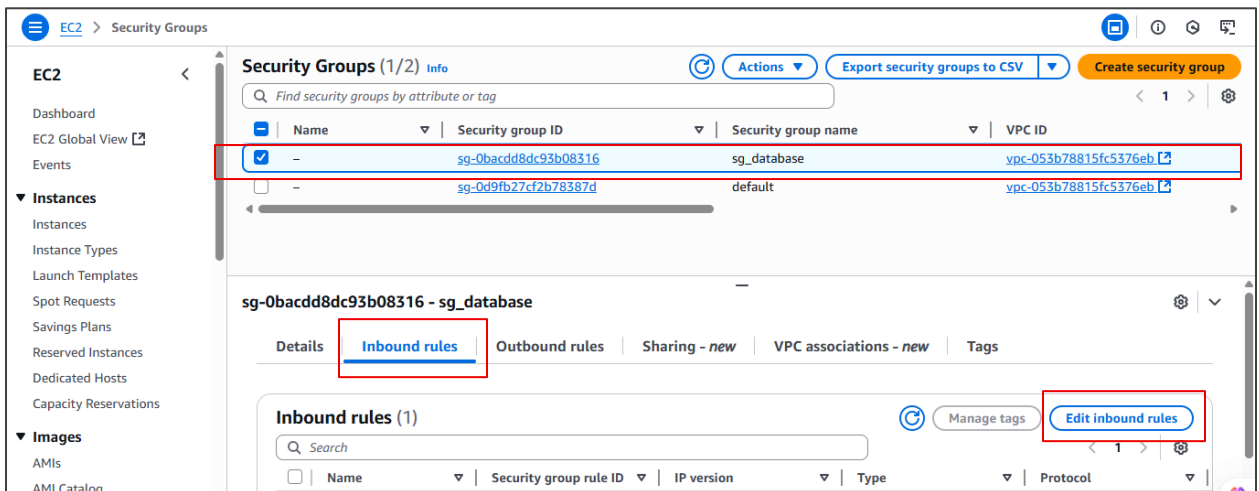
2.2 Click **Security groups**



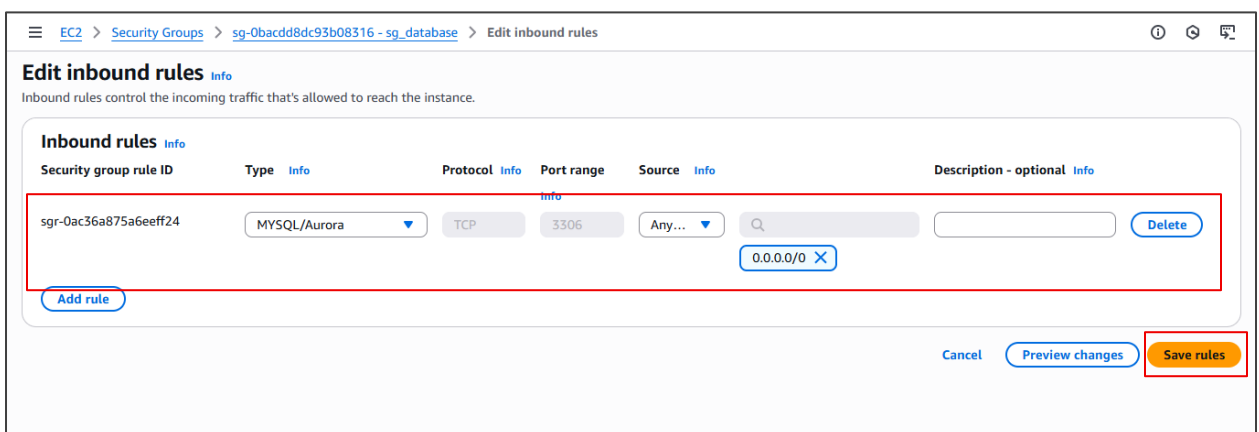
The following output page appears:



2.3 Select the **sg_database** security group, click **Inbound rules**, and click **Edit inbound rules** to configure access



2.4 Add an inbound rule to allow **MySQL/Aurora** traffic on port **3306** from **0.0.0.0/0**, then click **Save rules**

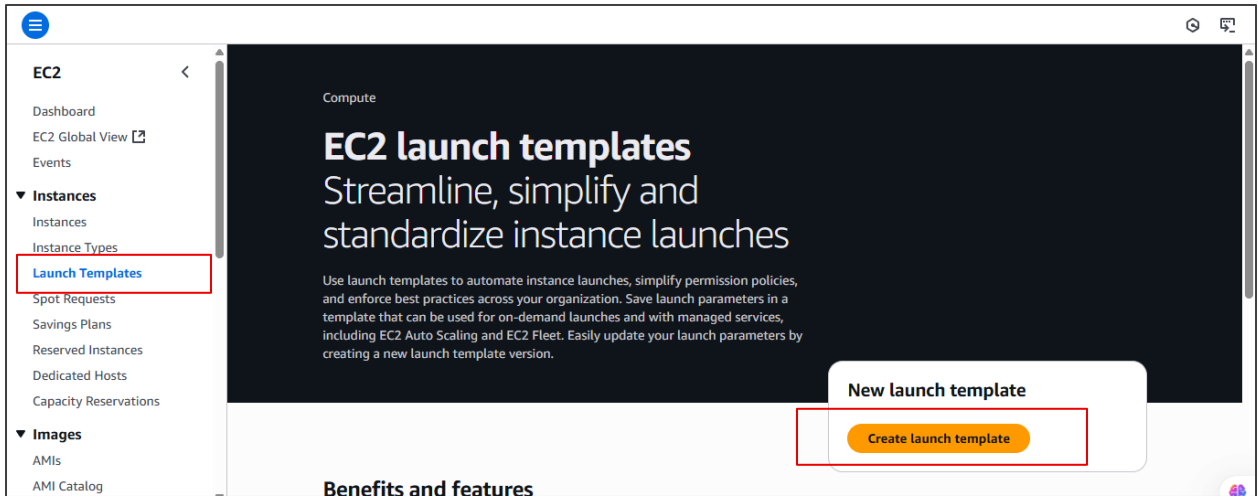


The following output page appears:

The screenshot shows the AWS Management Console interface for the 'Security Groups' page. The left sidebar contains navigation links for 'EC2', 'Dashboard', 'EC2 Global View', 'Events', and 'Instances'. The main content area is titled 'Security Groups (2)' and includes a search bar, a table of security groups, and a 'Create security group' button. The table lists two security groups: 'sg_database' and 'default'. The 'sg_database' group has a Security group ID of 'sg-0bacdd8dc93b08316' and is associated with VPC 'vpc-053b78815fc5376eb'. The 'default' group has a Security group ID of 'sg-0d9fb27cf2b78387d' and is also associated with VPC 'vpc-053b78815fc5376eb'.

Name	Security group ID	Security group name	VPC ID
sg_database	sg-0bacdd8dc93b08316	sg_database	vpc-053b78815fc5376eb
default	sg-0d9fb27cf2b78387d	default	vpc-053b78815fc5376eb

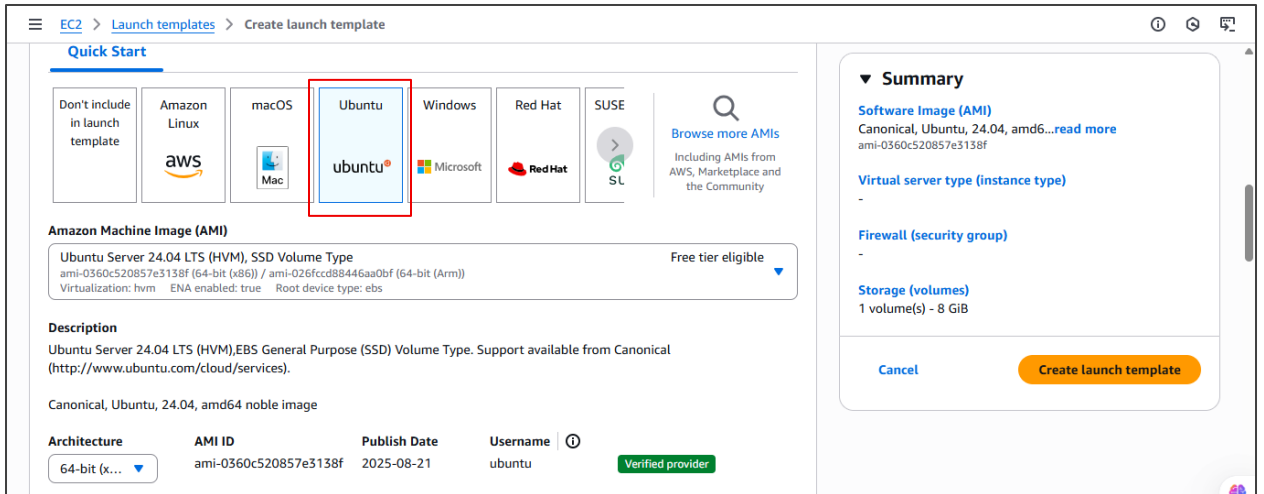
2.5 Click **Launch Templates**, then click **Create launch template** to start configuring a new EC2 launch template



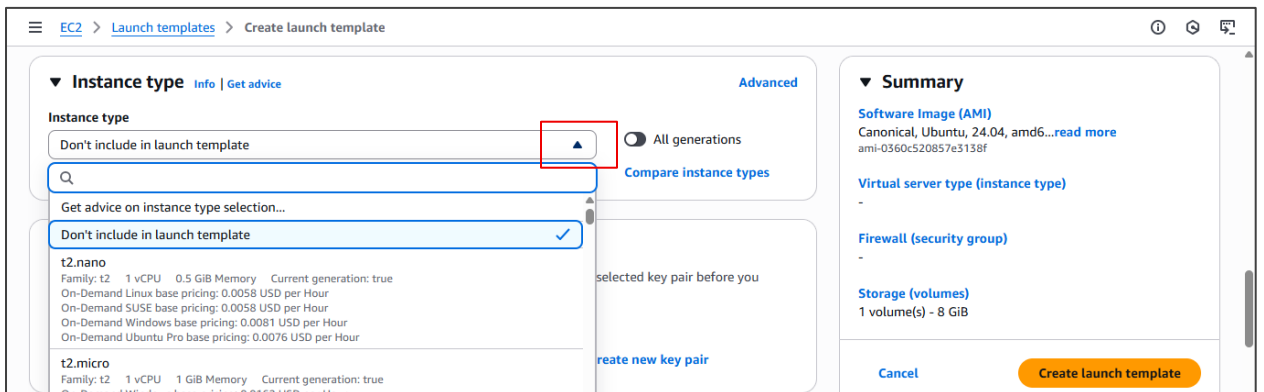
2.6 Enter **Launch template name** as **LaunchTemplate** and **Template version description** as **Launch Template for PHP Application** in the create launch template form

The screenshot displays the 'Create launch template' form in the AWS Management Console. The form is titled 'Create launch template' and includes a brief explanation of what a launch template is. The main section, 'Launch template name and description', contains two input fields: 'Launch template name - required' with the value 'LaunchTemplate' and 'Template version description' with the value 'Launch Template for PHP Application'. Both fields are highlighted with red boxes. To the right, a 'Summary' section lists the configuration options: 'Software Image (AMI)', 'Virtual server type (instance type)', 'Firewall (security group)', and 'Storage (volumes)'. At the bottom right, there are 'Cancel' and 'Create launch template' buttons.

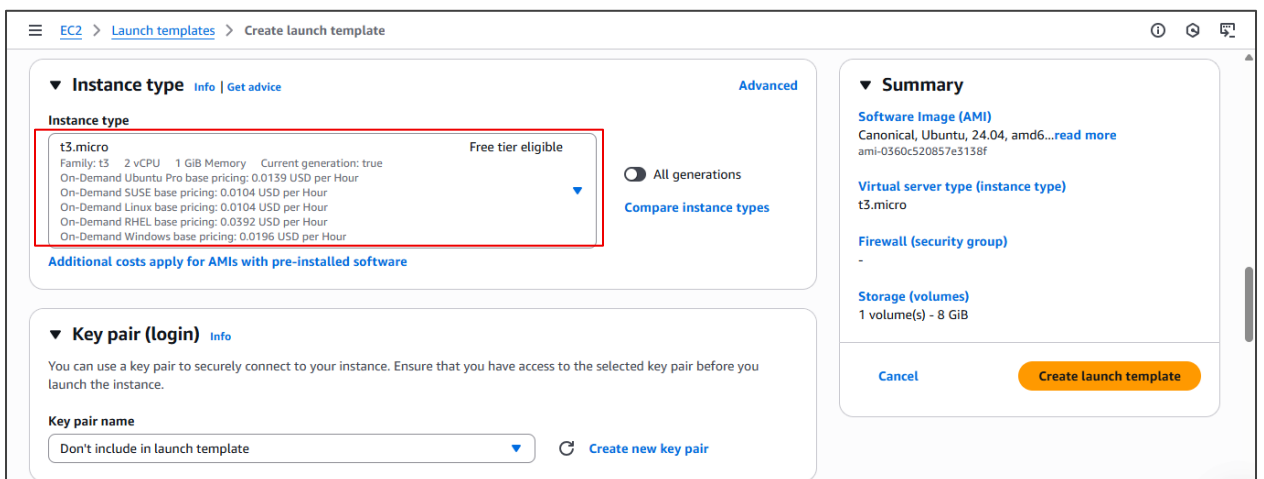
2.7 Scroll down and select **Ubuntu** as the Amazon Machine Image (AMI)



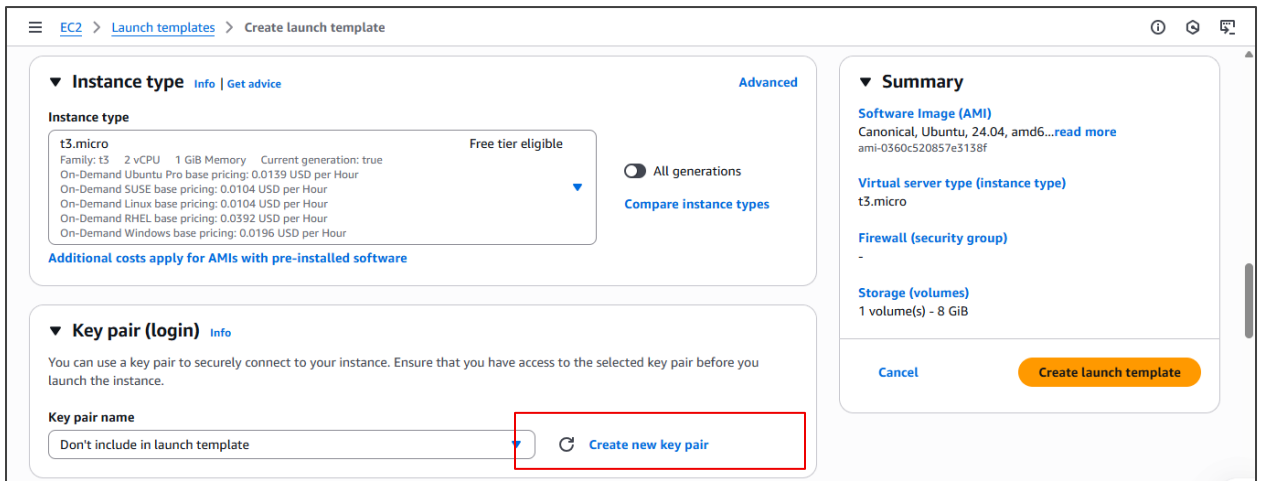
2.8 Scroll down and click the dropdown and select **t3.micro** instance type



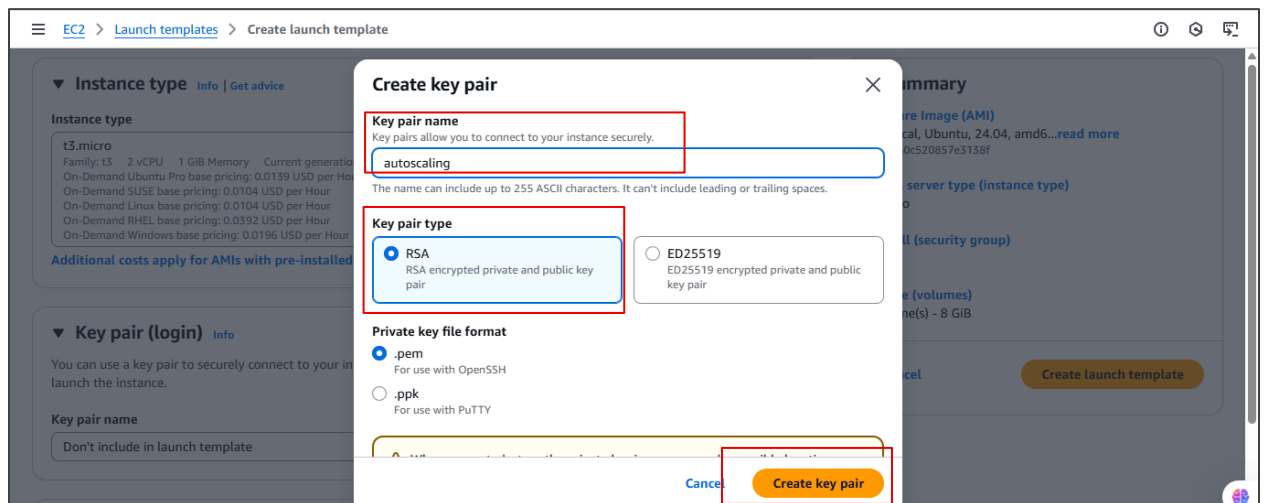
The following page appears:



2.9 Scroll down and click **Create a new key pair**



2.10 Give a **Key pair name** (for example, **autoscaling**), select **RSA** as the **Key pair type**, and then click **Create key pair**



2.11 Scroll down and select **Create security group**

EC2 > Launch templates > Create launch template

▼ Key pair (login) [Info](#)

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name
autoscaling ↕ [Create new key pair](#)

▼ Network settings [Info](#)

Subnet [Info](#)
Don't include in launch template ↕ [Create new subnet](#)

When you specify a subnet, a network interface is automatically added to your template.

Availability Zone [Info](#)
Don't include in launch template ↕ [Enable additional zones](#)

Firewall (security groups) [Info](#)
A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☐ Select existing security group ☒ Create security group

▼ Summary

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...[read more](#)
ami-0360c520857e3138f

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

2.12 Enter Security group name as sg_securityGroup and then Description as Security Group

EC2 > Launch templates > Create launch template

Security group name - required
sg_securityGroup

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-:/[!@#%&*~`]{0,255}

Description - required [Info](#)
Security Group

VPC [Info](#)
vpc-053b78815fc5376eb (default) ↕

Inbound Security Group Rules
No security group rules are currently included in this template. Add a new rule to include it in the launch template.

[Add security group rule](#)

[Advanced network configuration](#)

▼ Summary

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...[read more](#)
ami-0360c520857e3138f

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

2.13 Scroll down and click Add security group rule

EC2 > Launch templates > Create launch template

Security group name - required
sg_securityGroup

This security group will be added to all network interfaces. The name can't be edited after the security group is created. Max length is 255 characters. Valid characters: a-z, A-Z, 0-9, spaces, and _-:/[!@#%&*~`]{0,255}

Description - required [Info](#)
Security Group

VPC [Info](#)
vpc-053b78815fc5376eb (default) ↕

Inbound Security Group Rules
No security group rules are currently included in this template. Add a new rule to include it in the launch template.

[Add security group rule](#)

[Advanced network configuration](#)

▼ Summary

Software Image (AMI)
Canonical, Ubuntu, 24.04, amd64...[read more](#)
ami-0360c520857e3138f

Virtual server type (instance type)
t3.micro

Firewall (security group)
New security group

Storage (volumes)
1 volume(s) - 8 GiB

[Cancel](#) [Create launch template](#)

2.14 Set the following security group rules for the Launch templates:

Rule 1 (SSH): Select **Type** as **ssh**, **Protocol** as **TCP**, **Port range** as **22**, and **Source** as **0.0.0.0/0**

Rule 2 (HTTP): Select **Type** as **HTTP**, **Protocol** as **TCP**, **Port range** as **80**, and **Source** as **0.0.0.0/0**

2.15 Scroll down and click **Advanced details**

2.16 Copy the following user details, scroll down, and paste them into the **User data** option:

```
#!/bin/bash
set -e
# Update packages
apt update -y
apt upgrade -y
# Install Apache and PHP with MySQL support
apt install -y apache2 php libapache2-mod-php php-mysql
```

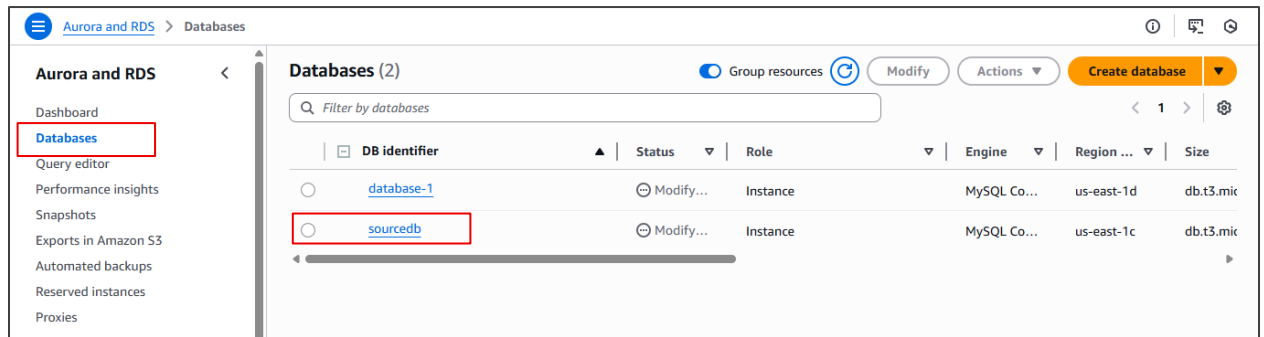
```

# Enable and start Apache service
systemctl enable apache2
systemctl start apache2
# RDS connection details
RDS_HOST="mydb.xxxxxx.us-east-1.rds.amazonaws.com"
RDS_USER="admin"
RDS_PASS="password"
DEFAULT_DB="mysql" # Default database to connect
# Deploy PHP app
cat <<EOF > /var/www/html/index.php
<?php
\$conn = mysqli_connect("$RDS_HOST", "$RDS_USER", "$RDS_PASS",
"$DEFAULT_DB");
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}
// Fetch all tables in default database
\$tables = mysqli_query(\$conn, "SHOW TABLES");
if (mysqli_num_rows(\$tables) > 0) {
    echo "<h1>Tables in database '$DEFAULT_DB':</h1><ul>";
    while (\$row = mysqli_fetch_row(\$tables)) {
        echo "<li>" . \$row[0] . "</li>";
    }
    echo "</ul>";
} else {
    echo "<h1>No tables found in database '$DEFAULT_DB'.</h1>";
}
mysqli_close(\$conn);
?>
EOF
# Set proper permissions
chown www-data:www-data /var/www/html/index.php
chmod 644 /var/www/html/index.php

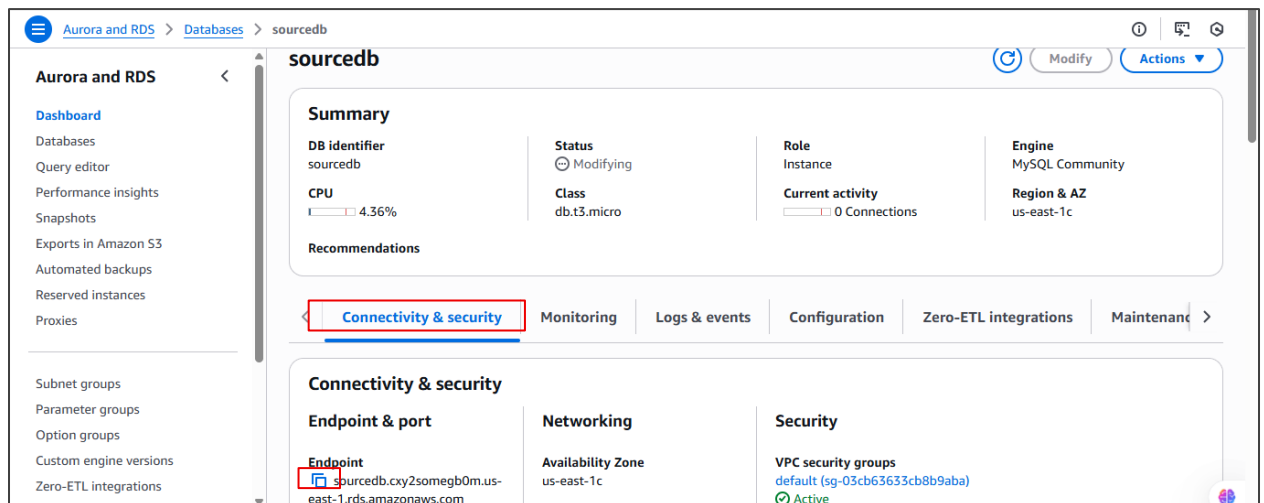
```



2.17 Navigate to **Databases** and click **sourcedb**



2.18 Click **Connectivity & security**, and click the copy endpoint button



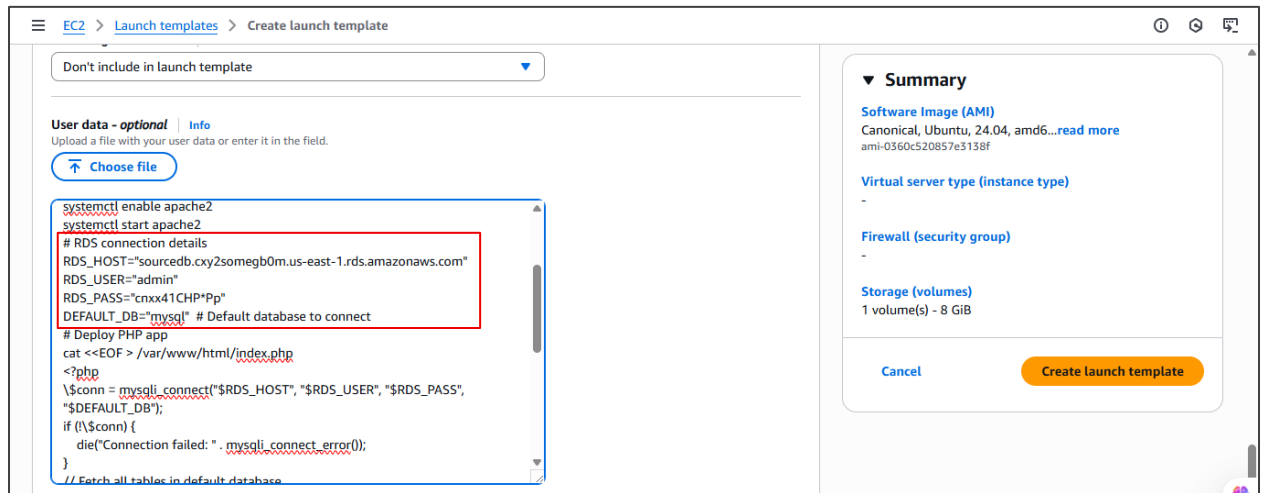
- 2.19 Update the RDS instance connection details from the user data as shown in the following image:

RDS_HOST="sourcedb.cxy2somegb0m.us-east-1.rds.amazonaws.com"

RDS_USER="admin"

RDS_PASS="cnxx41CHP*Pp"

DEFAULT_DB="mysql" # Default database to connect

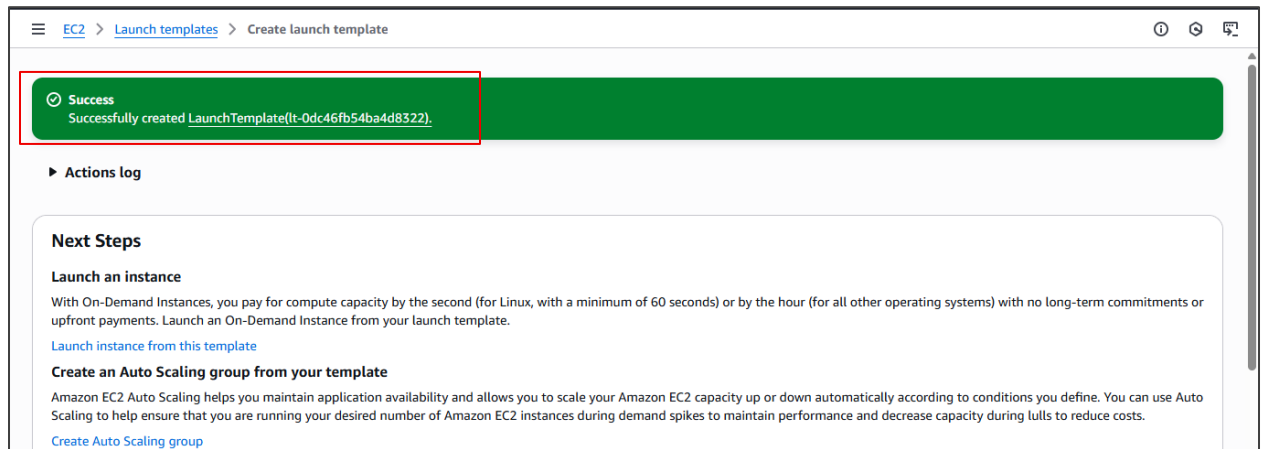


Note: Replace the value of RDS_HOST (e.g., mydb.xxxxxx.us-east-1.rds.amazonaws.com) with the copied endpoint from step 2.18 and the password with your actual one.

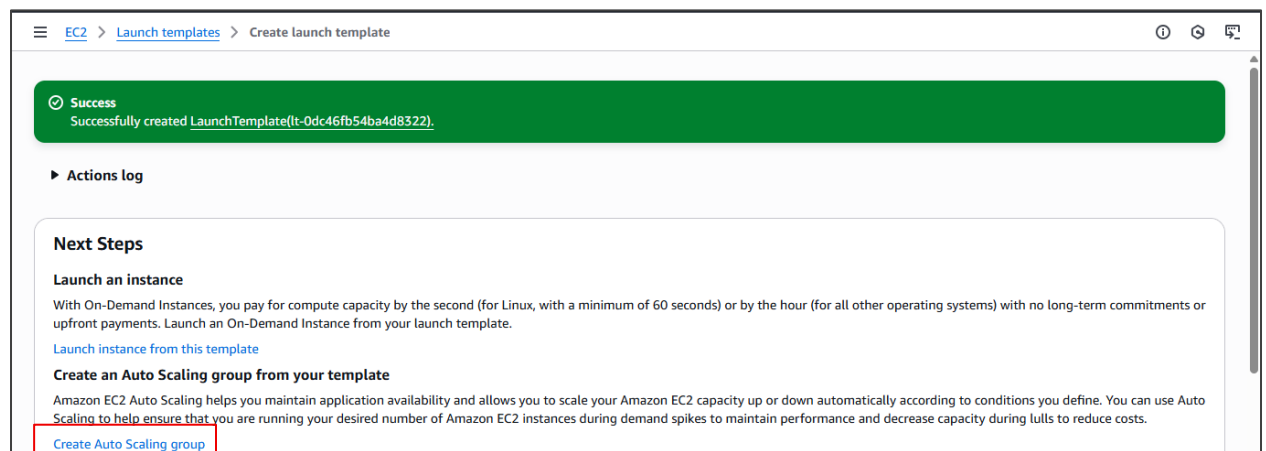
- 2.20 Click **Create launch template**



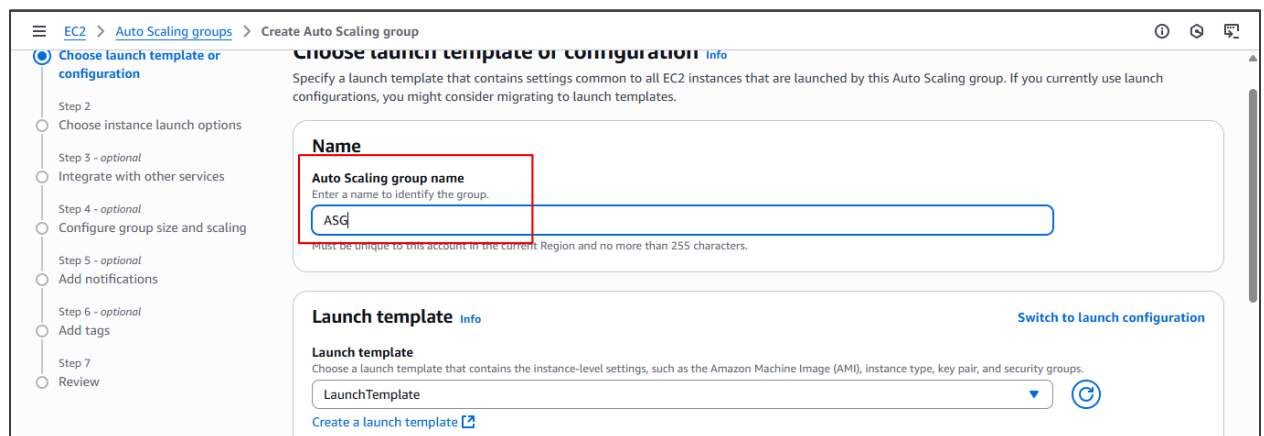
The following page appears:



2.21 Click **Create Auto Scaling group**



2.22 Enter **Auto Scaling group name** as **ASG**



2.23 Scroll down and click **Next**

EC2 > Auto Scaling groups > Create Auto Scaling group

Default (1) [Create a launch template version](#)

Description
Launch Template for PHP Application

Launch template
[LaunchTemplate](#)
lt-0dc46fb54ba4d8322

Instance type
t3.micro

AMI ID
ami-0360c520857e3138f

Security groups
-

Request Spot Instances
No

Key pair name
autoscaling

Security group IDs
[sg-02e90af3f21f8a3cf](#)

Additional details

Storage (volumes)
-

Date created
Sat Sep 27 2025 15:16:49 GMT+0530
(India Standard Time)

[Cancel](#) [Next](#)

2.24 Click the dropdown in the **Availability Zones and subnets** and select all the options

EC2 > Auto Scaling groups > Create Auto Scaling group

Choose the VPC that defines the virtual network for your Auto Scaling group.

vpc-053b78815fc5376eb
172.31.0.0/16 Default [Create a VPC](#)

Availability Zones and subnets
Define which Availability Zones and subnets your Auto Scaling group can use in the chosen VPC.

Select Availability Zones and subnets

- ☐ use1-az1 (us-east-1b) | subnet-084ba13dbc5ad705a
172.31.0.0/20 Default
- ☐ use1-az2 (us-east-1c) | subnet-078d1f6dc5c09882c
172.31.80.0/20 Default
- ☐ use1-az3 (us-east-1e) | subnet-0655808ddf3e2c185
172.31.48.0/20 Default
- ☐ use1-az4 (us-east-1d) | subnet-01054fa25f27c0c1a
172.31.16.0/20 Default
- ☐ use1-az5 (us-east-1f) | subnet-035f4158664a86d58
172.31.64.0/20 Default

Balanced only
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

[Cancel](#) [Skip to review](#) [Previous](#) [Next](#)

2.25 Click **Next**

EC2 > Auto Scaling groups > Create Auto Scaling group

use1-az5 (us-east-1f) | subnet-035f4158664a86d58
172.31.64.0/20 Default

use1-az6 (us-east-1a) | subnet-0027162d92f66d181
172.31.32.0/20 Default

[Create a subnet](#)

Availability Zone distribution - new
Auto Scaling automatically balances instances across Availability Zones. If launch failures occur in a zone, select a strategy.

☒ **Balanced best effort**
If launches fail in one Availability Zone, Auto Scaling will attempt to launch in another healthy Availability Zone.

☐ **Balanced only**
If launches fail in one Availability Zone, Auto Scaling will continue to attempt to launch in the unhealthy Availability Zone to preserve balanced distribution.

⚠ Your requested instance type (t3.micro) is not available in 1 Availability Zone. You may need to change the instance type or choose other Availability Zones for better resiliency. [Learn more](#)

Cancel Skip to review Previous **Next**

2.26 Select **Attach to a new load balancer**, then in the Attach to a new load balancer section, choose **Application Load Balancer (HTTP, HTTPS)** as the **Load balancer type**, enter **ASG-1** as the **Load balancer name**, and select **Internet-facing** as the **Load balancer scheme**

EC2 > Auto Scaling groups > Create Auto Scaling group

Step 4 - optional
● Configure group size and scaling

Step 5 - optional
● Add notifications

Step 6 - optional
● Add tags

Step 7
● Review

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☐ Attach to an existing load balancer
Choose from your existing load balancers.

☒ **Attach to a new load balancer**
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to a new load balancer
Define a new load balancer to create for attachment to this Auto Scaling group.

Load balancer type
Choose from the load balancer types offered below. Type selection cannot be changed after the load balancer is created. If you need a different type of load balancer than those offered here, visit the [Load Balancing console](#).

☒ **Application Load Balancer**
HTTP, HTTPS

☐ Network Load Balancer
TCP, UDP, TLS

Load balancer name
Name cannot be changed after the load balancer is created.

ASG-1

Load balancer scheme
Scheme cannot be changed after the load balancer is created.

☐ Internal

☒ **Internet-facing**

2.27 Scroll down and click **Next**

EC2 > Auto Scaling groups > Create Auto Scaling group

instance replacement occurs.

EC2 health checks
[Always enabled](#)

Additional health check types - optional [Info](#)

- ☐ Turn on Elastic Load Balancing health checks **Recommended**
Elastic Load Balancing monitors whether instances are available to handle requests. When it reports an unhealthy instance, EC2 Auto Scaling can replace it on its next periodic check.
- ☒ Turn on VPC Lattice health checks
VPC Lattice can monitor whether instances are available to handle requests. If it considers a target as failed a health check, EC2 Auto Scaling replaces it after its next periodic check.
- ☐ Turn on Amazon EBS health checks
EBS monitors whether an instance's root volume or attached volume stalls. When it reports an unhealthy volume, EC2 Auto Scaling can replace the instance on its next periodic health check.

Health check grace period [Info](#)
This time period delays the first health check until your instances finish initializing. It doesn't prevent an instance from terminating when placed into a non-running state.

300 seconds

[Cancel](#) [Skip to review](#) [Previous](#) **Next**

2.28 Click the dropdown, select **Create a target group**, and give the name as **target**

EC2 > Auto Scaling groups > Create Auto Scaling group

☒ use1-az2 (us-east-1c) subnet-078d1f6dc5c09882c

☒ use1-az1 (us-east-1b) subnet-084ba13dbc5ad705a

☒ use1-az6 (us-east-1a) subnet-0027162d92f66d181

Listeners and routing
If you require secure listeners, or multiple listeners, you can configure them from the [Load Balancing console](#) after your load balancer is created.

Protocol HTTP **Port** 80

Default routing (forward to)
Select new or existing target group

Create a target group

Tags - optional
Consider adding tags to your load balancer. Tags enable you to categorize your AWS resources so you can more easily manage them.

EC2 > Auto Scaling groups > Create Auto Scaling group

☒ use1-az2 (us-east-1c) subnet-078d1f6dc5c09882c

☒ use1-az1 (us-east-1b) subnet-084ba13dbc5ad705a

☒ use1-az6 (us-east-1a) subnet-0027162d92f66d181

Listeners and routing
If you require secure listeners, or multiple listeners, you can configure them from the [Load Balancing console](#) after your load balancer is created.

Protocol HTTP **Port** 80

Default routing (forward to)
Create a target group

New target group name
An instance target group with default settings will be created.

target

Tags - optional
Consider adding tags to your load balancer. Tags enable you to categorize your AWS resources so you can more easily manage them.

[Add tag](#)
50 remaining

2.29 Scroll down and click **Next**

The screenshot shows the 'Create Auto Scaling group' page in the AWS Management Console. The breadcrumb navigation at the top indicates the path: EC2 > Auto Scaling groups > Create Auto Scaling group. The main content area is titled 'Health checks' and includes a link to 'Always enabled'. Below this, there is a section for 'Additional health check types - optional' with three checkboxes: 'Turn on Elastic Load Balancing health checks' (marked as 'Recommended'), 'Turn on VPC Lattice health checks', and 'Turn on Amazon EBS health checks'. Each checkbox has a brief description of its function. At the bottom of this section, there is a 'Health check grace period' field set to '300 seconds'. On the right side of the page, there are four buttons: 'Cancel', 'Skip to review', 'Previous', and 'Next'. The 'Next' button is highlighted with a red rectangular box.

2.30 Select **Target tracking scaling policy**

The screenshot shows the 'Create Auto Scaling group' page in the AWS Management Console, specifically the 'Scaling' section. The breadcrumb navigation at the top indicates the path: EC2 > Auto Scaling groups > Create Auto Scaling group. On the left side, there is a sidebar with a list of steps: 'Step 6 - optional', 'Add tags', 'Step 7', and 'Review'. The main content area is titled 'Scaling' and includes a link to 'Info'. Below this, there is a section for 'Scaling limits' with two input fields: 'Min desired capacity' and 'Max desired capacity', both set to '1'. At the bottom of this section, there is a section for 'Automatic scaling - optional' with a link to 'Info'. Below this, there is a section for 'Choose whether to use a target tracking policy' with two radio buttons: 'No scaling policies' and 'Target tracking scaling policy'. The 'Target tracking scaling policy' radio button is selected and highlighted with a red rectangular box.

2.31 Scroll down and click **Next**

The screenshot shows the 'Create Auto Scaling group' wizard in the AWS Management Console. The breadcrumb navigation at the top indicates the path: EC2 > Auto Scaling groups > Create Auto Scaling group. The main content area is titled 'Additional settings' and contains three sections: 'Instance scale-in protection' with a checkbox for 'Enable instance scale-in protection'; 'Monitoring' with a checkbox for 'Enable group metrics collection within CloudWatch'; and 'Default instance warmup' with a checkbox for 'Enable default instance warmup'. At the bottom right, there are four buttons: 'Cancel', 'Skip to review', 'Previous', and 'Next'. The 'Next' button is highlighted with a red rectangular box.

2.32 Click **Next**

The screenshot shows the 'Create Auto Scaling group' wizard in the AWS Management Console, specifically Step 5: 'Add notifications - optional'. The left sidebar shows a list of steps: Step 1 (Choose launch template or configuration), Step 2 (Choose instance launch options), Step 3 - optional (Integrate with other services), Step 4 - optional (Configure group size and scaling), Step 5 - optional (Add notifications), and Step 6 - optional (Add tags). Step 5 is currently selected. The main content area is titled 'Add notifications - optional' and includes a description: 'Send notifications to SNS topics whenever Amazon EC2 Auto Scaling launches or terminates the EC2 instances in your Auto Scaling group.' Below this is an 'Add notification' button. At the bottom right, there are four buttons: 'Cancel', 'Skip to review', 'Previous', and 'Next'. The 'Next' button is highlighted with a red rectangular box.

2.33 Click **Next**

The screenshot shows the 'Create Auto Scaling group' wizard in the AWS Management Console, specifically Step 6: 'Add tags - optional'. The left sidebar shows a list of steps: Step 1 (Choose launch template or configuration), Step 2 (Choose instance launch options), Step 3 - optional (Integrate with other services), Step 4 - optional (Configure group size and scaling), Step 5 - optional (Add notifications), Step 6 - optional (Add tags), and Step 7 (Review). Step 6 is currently selected. The main content area is titled 'Add tags - optional' and includes a description: 'Add tags to help you search, filter, and track your Auto Scaling group across AWS. You can also choose to automatically add these tags to instances when they are launched.' Below this is a 'Tags (0)' section with an 'Add tag' button and a note '50 remaining'. At the bottom right, there are four buttons: 'Cancel', 'Previous', and 'Next'. The 'Next' button is highlighted with a red rectangular box.

2.34 Click **Create Auto Scaling group**

EC2 > Auto Scaling groups > Create Auto Scaling group

Preference: Default, Capacity reservation: -, Resource Groups: -

Step 5: Add notifications [Edit](#)

Notifications
No notifications

Step 6: Add tags [Edit](#)

Tags (0)

Key	Value
Tag new instances	

No tags

[Preview code](#) [Cancel](#) [Previous](#) [Create Auto Scaling group](#)

The following output page appears:

EC2 > Auto Scaling groups

Auto Scaling groups (1) [Info](#) Last updated less than a minute ago [Launch configurations](#) [Launch templates](#) [Actions](#) [Create Auto Scaling group](#)

Search your Auto Scaling groups

☐	Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max	Availability
☐	ASG	LaunchTemplate Version Default	0	Updating capacity...	1	1	1	6 Availabili

Step 3: Validate Auto Scaling and Multi-AZ mechanism

3.1 Click **Target Groups**

Lifecycle Manager

Network & Security

- Security Groups
- Elastic IPs
- Placement Groups
- Key Pairs
- Network Interfaces

Load Balancing

- Load Balancers
- Target Groups**
- Trust Stores

Resources [EC2 Global View](#) [Settings](#) [Refresh](#)

You are using the following Amazon EC2 resources in the United States (N. Virginia) Region:

Instances (running)	0	Auto Scaling Groups	1
Capacity Reservations	API Error	Dedicated Hosts	API Error
Elastic IPs	0	Instances	0
Key pairs	1	Load balancers	1
Placement groups	0	Security groups	3
Snapshots	0	Volumes	0

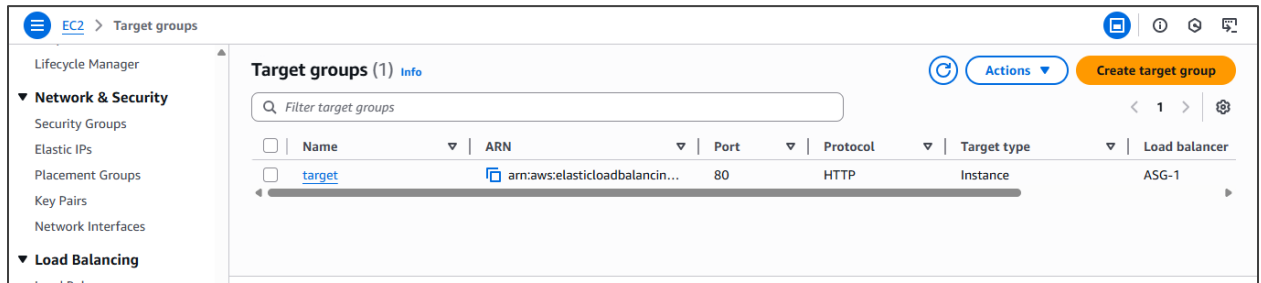
Account attributes [Refresh](#)

Settings

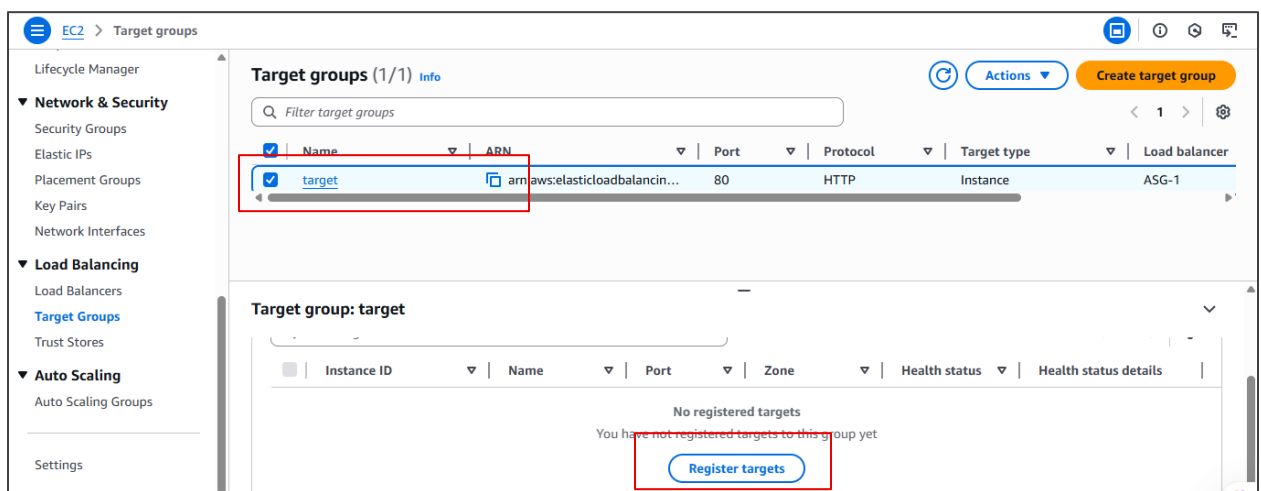
- Data protection and security
- Allowed AMIs
- Zones
- EC2 Serial Console
- Default credit specification
- EC2 console preferences

Explore AWS [Close](#)

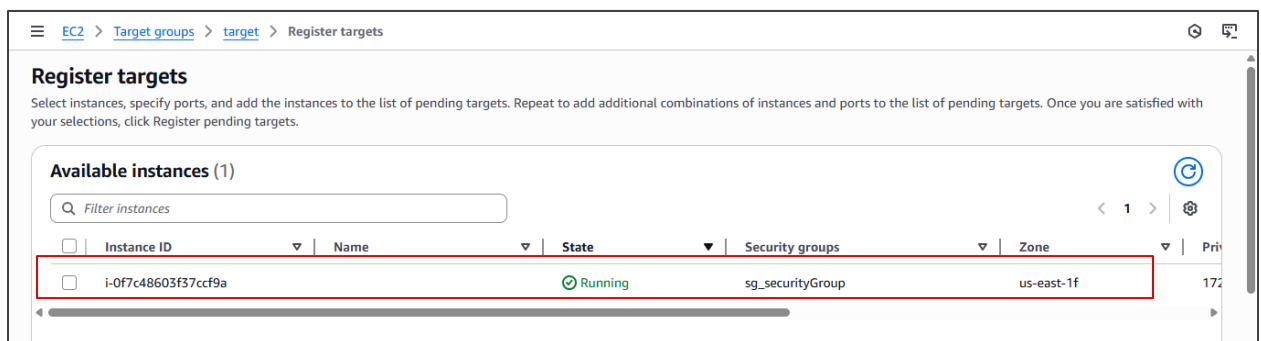
The following page appears:



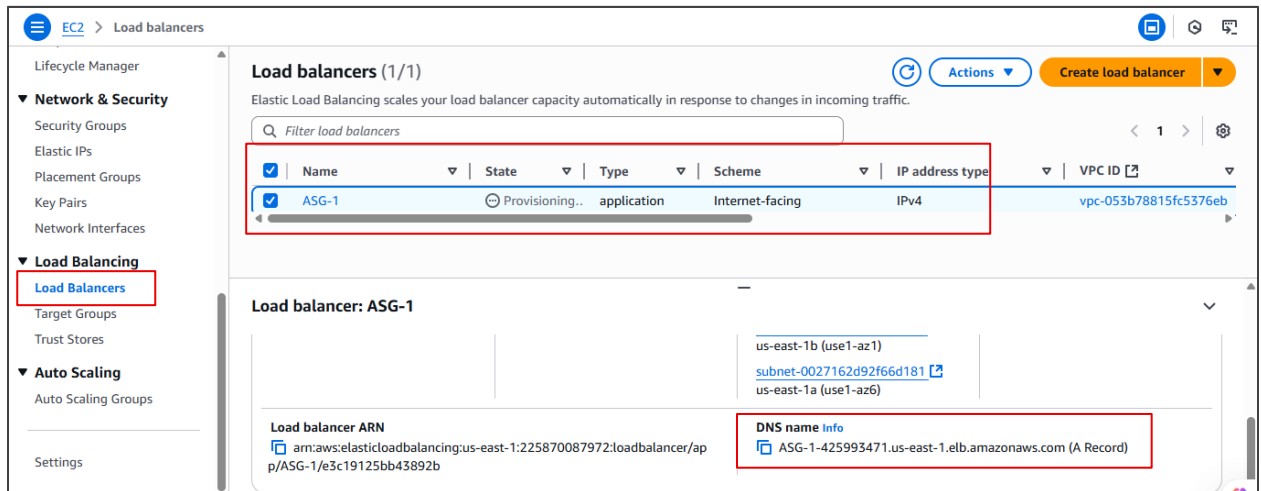
3.2 Select the **target** and click **Register targets**



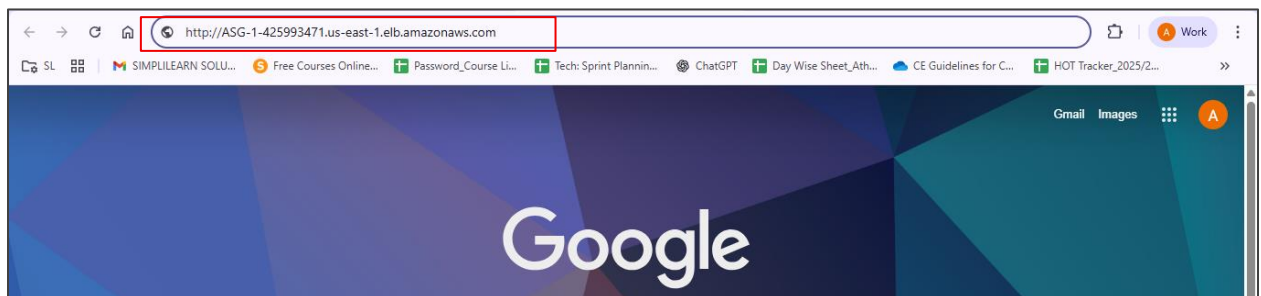
The following page appears:



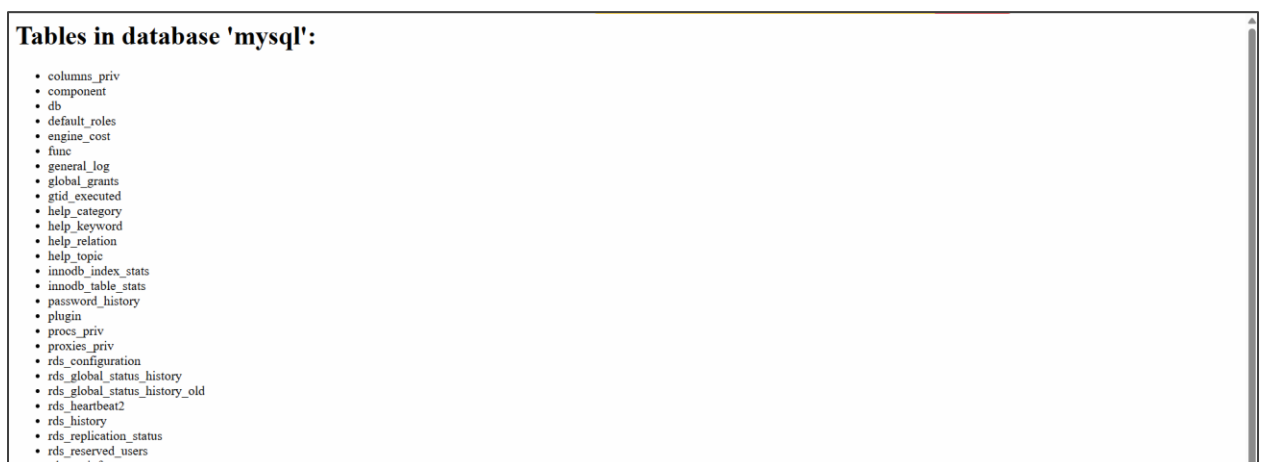
3.3 Click Load Balancers, click **ASG-1**, and then copy the **DNS name**



3.4 Paste the copied DNS name link in a new tab and press the enter key

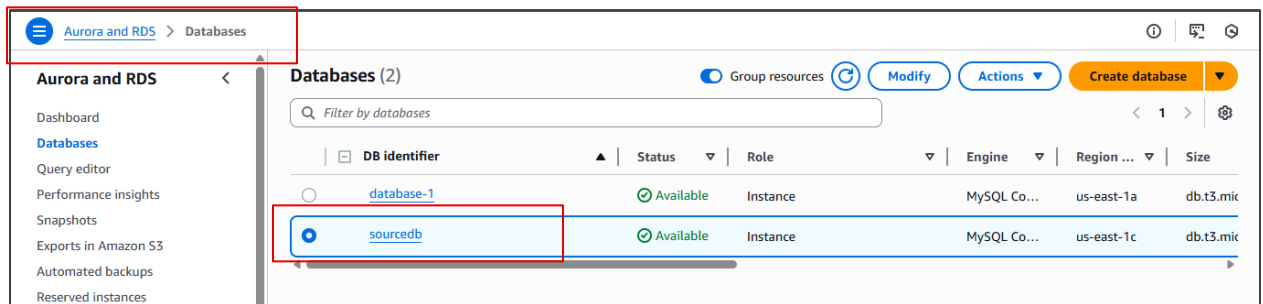


The following output page appears:



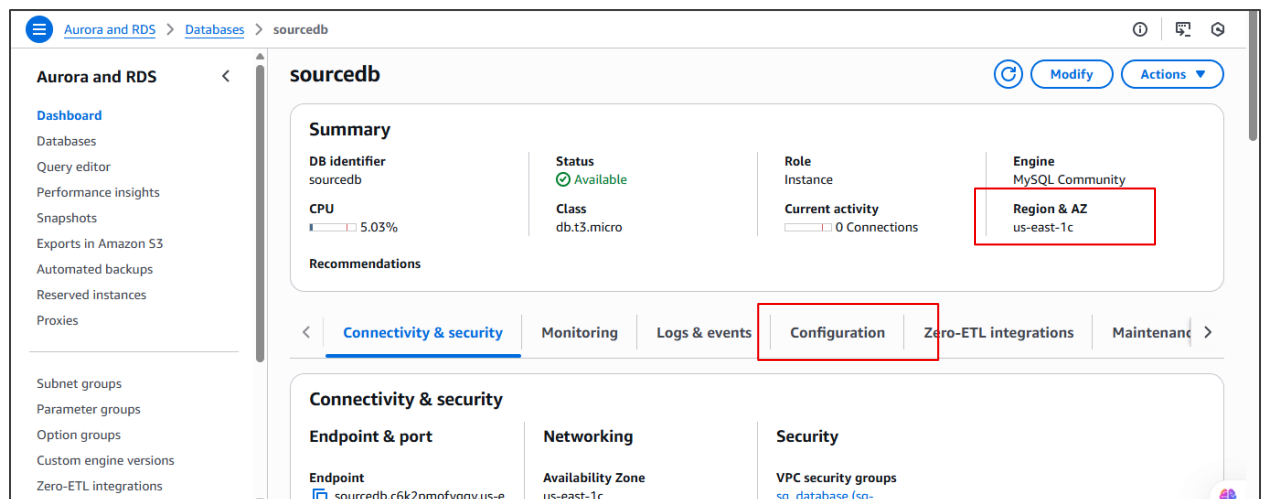
Note: When you access the ALB DNS name in your browser, the PHP page will connect to the RDS instance using the credentials defined in the user data. If the connection is successful, it will display a list of tables from the database specified in DEFAULT_DB. The exact output can vary depending on the database selected and the tables available in it, but seeing such a result confirms that the EC2 instances can reach the RDS instance and that the PHP web page is correctly configured to fetch and display database tables.

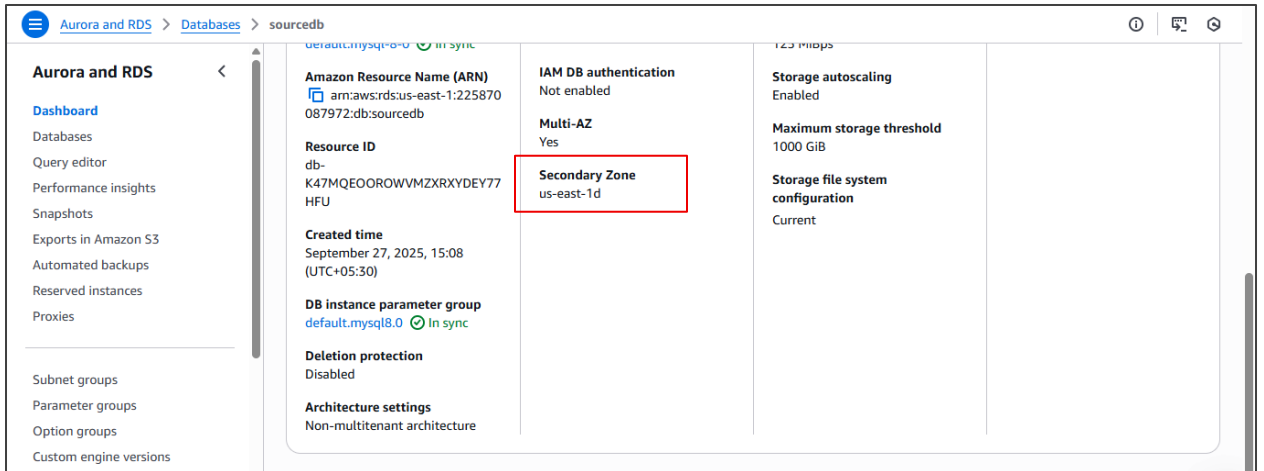
3.5 Navigate to **Aurora and RDS Databases** and click sourcedb



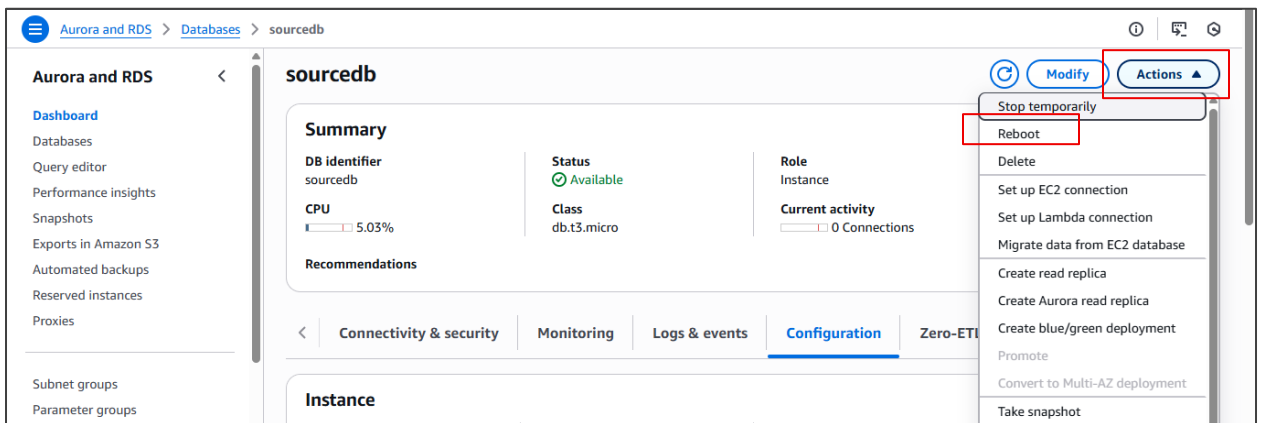
The following output appears as the current primary availability zone is **us-east-1c**, and the Secondary Zone is **us-east-1d**:

Note: Click Configuration to check the Secondary Zone

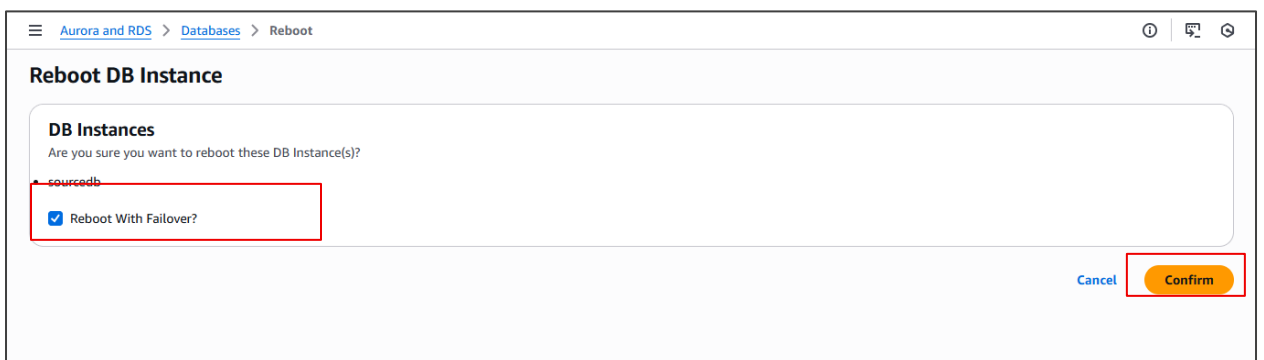




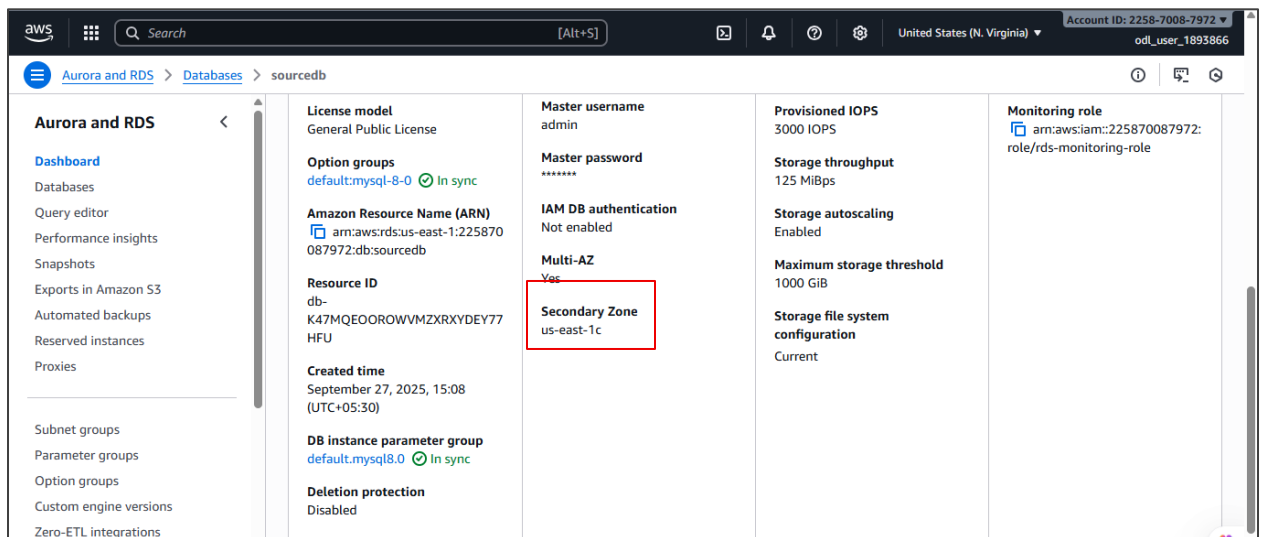
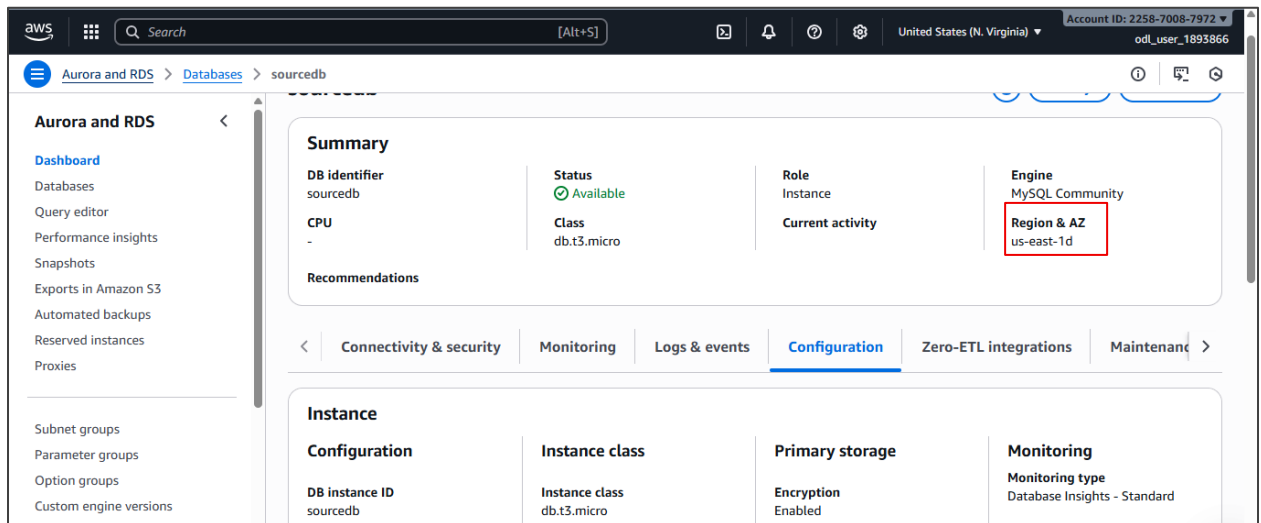
3.6 Click **Actions** and select **Reboot**



3.7 Tick the checkbox and click **Confirm**



The following output appears as the current primary availability zone is **us-east-1d** and the Secondary Zone is **us-east-1c**:



Note: After a restart or failover, the roles of the availability zones switch. For example, us-east-1d may become the new primary, and us-east-1c will act as the secondary. This behavior is part of RDS's Multi-AZ high availability design, ensuring minimal downtime and continuous availability.

By following these steps, you deployed a resilient, internet-facing web application behind an ALB, validated MySQL connectivity from the PHP page using the RDS endpoint, and enabled Auto Scaling with Multi-AZ RDS for high availability. The app is

reachable via the ALB DNS name, scales under load, and survives AZ failovers with minimal disruption. You also enforced the correct security group rules, ensuring that only HTTP/SSH and DB traffic flows as intended. Next, harden credentials/SSL, add health checks and alarms, and automate rollouts to take this from demo to production.